

Completed Solidity Auction Assignment

The missing sections (Tasks 1–6 and the Part 2 modifier) have been completed below.

Completed Code

```
pragma solidity ^0.4.17;

contract Auction {

    struct Item {
        uint itemId;
        uint[] itemTokens;
    }

    struct Person {
        uint remainingTokens;
        uint personId;
        address addr;
    }

    mapping(address => Person) tokenDetails;
    Person [4] bidders;

    Item [3] public items;
    address[3] public winners;
    address public beneficiary;

    uint bidderCount = 0;

    function Auction() public payable {

        // Part 1 Task 1
        beneficiary = msg.sender;

        uint[] memory emptyArray;
        items[0] = Item({itemId:0,itemTokens:emptyArray});

        // Part 1 Task 2
        items[1] = Item({itemId:1,itemTokens:emptyArray});
        items[2] = Item({itemId:2,itemTokens:emptyArray});
    }

    function register() public payable {

        bidders[bidderCount].personId = bidderCount;

        // Part 1 Task 3
        bidders[bidderCount].addr = msg.sender;

        bidders[bidderCount].remainingTokens = 5;
        tokenDetails[msg.sender] = bidders[bidderCount];
        bidderCount++;
    }

    function bid(uint _itemId, uint _count) public payable {

        // Part 1 Task 4
        require(tokenDetails[msg.sender].remainingTokens >= _count);
        require(tokenDetails[msg.sender].remainingTokens > 0 && _itemId <= 2);

        // Part 1 Task 5
        uint balance = tokenDetails[msg.sender].remainingTokens - _count;

        tokenDetails[msg.sender].remainingTokens = balance;
        bidders[tokenDetails[msg.sender].personId].remainingTokens = balance;

        Item storage bidItem = items[_itemId];
        for(uint i=0; i<_count; i++) {
            bidItem.itemTokens.push(tokenDetails[msg.sender].personId);
        }
    }

    // Part 2 Task 1
    modifier onlyOwner {
```

```

        require(msg.sender == beneficiary);
    -;
}

function revealWinners() public onlyOwner {
    for (uint id = 0; id < 3; id++) {
        Item storage currentItem = items[id];

        if(currentItem.itemTokens.length != 0) {

            uint randomIndex =
                (block.number / currentItem.itemTokens.length)
                % currentItem.itemTokens.length;

            uint winnerId = currentItem.itemTokens[randomIndex];

            // Part 1 Task 6
            winners[id] = bidders[winnerId].addr;
        }
    }
}

function getPersonDetails(uint id) public constant returns(uint,uint,address) {
    return (
        bidders[id].remainingTokens,
        bidders[id].personId,
        bidders[id].addr
    );
}
}

```